Agentic AI and LLMs in Fuzz Testing and Harness Development

Keegan Ravindran
Singapore Management University
keeganr.2023@scis.smu.edu.sg

September 2, 2026

Abstract

Abstract. Fuzz testing is a cornerstone of software security, yet its scalability is severely hindered by the manual effort required to develop effective harnesses, particularly for complex, stateful network protocols. While Large Language Models (LLMs) offer a potential solution, standard generative approaches often fail when applied to monolithic systems like the FR-Routing (FRR) suite due to hallucinations and architectural isolation requirements. This paper investigates the efficacy of Agentic AI in automating harness development for such environments. We propose a novel "Embedded Fuzzing Block" strategy, adapting the *Light-Fuzz-Gen* framework to inject fuzzing logic directly into the target daemon's runtime, thereby preserving essential global state and initialization sequences. Our agentic workflow—integrating context mining, heuristic daemon detection, and hybrid template generation—was evaluated against manual baselines across four core FRR daemons. The experimental results demonstrate that our approach achieved a total project line coverage of 7.61%, surpassing the 7.04% runtime coverage currently reported by OSS-Fuzz. These findings indicate that Agentic AI can effectively serve as a force multiplier in security auditing, enabling the automated discovery of vulnerabilities in critical infrastructure that remain unreachable by traditional manual harnessing.

1 Introduction

1.1 Background

Software vulnerabilities remain one of the leading causes of security breaches in modern software systems. Fuzzing has emerged as a valuable technique for uncovering such vulnerabilities by subjecting software to randomized or mutated inputs and observing unexpected behavior. In the past decade, fuzzing has evolved from simple input generation toward coverage-guided and grammar-aware approaches, achieving significant success in uncovering vulnerabilities across diverse domains.

Recent years have seen growing interest in applying Artificial Intelligence (AI) to this domain. Specifically, *Agentic AI* refers to systems that exhibit autonomous, goal-directed behavior, adapting to real-time responses and performing multi-step tasks with minimal human intervention. Unlike traditional static heuristics, agentic AI can iteratively refine its strategies in response to feedback, making it highly suitable for domains such as fuzzing where adaptation and exploration are essential.

1.2 Problem Statement

A crucial aspect of fuzzing is the *fuzz driver* or *harness*—a lightweight program that connects the fuzzer to the target code or library under testing. These harnesses define how inputs are consumed by the program and ensure that test cases meaningfully exercise the code. However, writing effective harnesses notoriously requires significant manual effort and expert knowledge of target APIs and input constraints. This bottleneck limits the scalability and automation of fuzzing, especially for large-scale open-source projects.

While recent studies (e.g., Zhang et al., 2023) have shown that Large Language Models (LLMs) can generate fuzz drivers, these efforts expose critical limitations. LLMs often struggle with complex APIs, stateful initialization, and semantic correctness when operating in a "single-pass" generation mode. Consequently, there is a need for workflows that can autonomously correct and refine these harnesses to reduce manual toil and improve reliability.

1.3 Research Objectives

The primary objectives of this research project are:

1. **Goal 1:** Conduct a literature review to assess the current status of Agentic AI's application in improving the performance of fuzzing (e.g. seed and harness generation).
2. **Goal 2:** Apply fuzzing with existing LLM-based methods for Fuzz Harness Generation to identify security vulnerabilities and improve fuzzing efforts.
3. **Goal 3:** Evaluate the effectiveness of LLM generated fuzz harnesses in a target project (FRRouting).

2 Literature Review

The integration of Large Language Models (LLMs) into software testing has precipitated a foundational shift in fuzzing architectures. The period spanning 2024 to 2025 marks a transition from "Generative Fuzzing"—where models stochastically predict inputs or drivers—to "Agentic Fuzzing," characterized by autonomous systems capable of reasoning, tool utilization, and iterative self-correction. This section analyzes this evolution across four critical domains: automated harness generation, false positive reduction, black-box analysis, and stateful protocol exploration.

2.1 From Generative to Iterative Harness Development

The efficacy of library fuzzing is bounded by the quality of the *fuzz driver* (harness). Early applications of LLMs treated harness generation as a translation task, mapping function signatures to C/C++ code. However, foundational studies by Zhang et al. [8] revealed significant limitations in this "single-pass" approach. Their empirical evaluation demonstrated that while LLMs possess the semantic knowledge to generate syntactically correct code, they struggle with complex API contracts without external grounding. Crucially, Zhang et al. identified that *iterative querying*—feeding compilation errors back to the model—yielded the highest performance gains, effectively transforming the LLM from a generator into a debugger.

2.2 Agentic Constraints and False Positive Reduction

As LLM-based generation moved from academic prototypes to industrial frameworks (e.g., Google's OSS-Fuzz-Gen), the challenge shifted from compilability to semantic correctness. A major bottleneck identified in recent literature is the proliferation of "false positive" crashes—drivers that crash due to misuse of the API rather than underlying library bugs.

To address this, the *FalseCrashReducer* framework [1] introduced a dual-agent architecture comprising proactive and reactive strategies:

- **Proactive Constraint Enforcement:** A "Function Analyzer" agent extracts preconditions (e.g., buffer size constraints) before generation, injecting them into the prompt to prevent naive API misuse.
- **Reactive Crash Validation:** A "Validator" agent analyzes stack traces from reported crashes to determine feasibility from valid entry points.

This "Synthesizer-Validator" loop has become a recurring motif in high-performing agentic systems, distinguishing them from earlier heuristic approaches.

2.3 Black-Box Analysis and Tool Usage

While harness generation typically assumes access to source code (white-box), recent advancements have extended agentic capabilities to binary-only environments. *LibLMFuzz* (2025) [4] represents a breakthrough in this domain, functioning as a middleware that interfaces LLMs with reverse engineering tools.

Unlike standard generation, LibLMFuzz employs an agentic workflow where the model autonomously queries ‘radare2‘ to disassemble functions and infer signatures before attempting code generation. By creating a feedback loop between the disassembler, the compiler, and the execution engine, LibLMFuzz achieved 100% API coverage across tested libraries, effectively commoditizing the reverse engineering process required for black-box auditing.

2.4 Directed Fuzzing and Predicate Synthesis

Beyond driver generation, Agentic AI has been applied to the "reachability" problem—guiding fuzzers to specific, hard-to-reach code paths. Traditional Directed Greybox Fuzzing (DGF) relies on distance metrics, which often fail in the face of complex path conditions (e.g., magic byte checks).

The *Locus* framework [9] introduced the concept of *Agentic Predicate Synthesis*. Rather than generating inputs directly, Locus uses agents to synthesize intermediate conditional logic (predicates) that act as "breadcrumbs" for the underlying fuzzer. By modifying the target binary to be "more fuzzable," Locus achieved a 41.6x average speedup in reachability compared to state-of-the-art

baselines. This highlights the capacity of agents to reason about control flow and data dependencies, a capability absent in traditional mutation-based fuzzers.

2.5 Stateful Protocol Fuzzing via RAG

Network protocols present unique challenges due to their stateful nature, where vulnerabilities often lie deep within a sequence of message transitions. Early LLM attempts, such as ChatAFL, struggled with the limited context window required to memorize complex RFC specifications.

To mitigate this, *MultiFuzz* [5] integrated Retrieval-Augmented Generation (RAG) into a multi-agent system. By embedding RFC documentation into a vector database, MultiFuzz allows agents to retrieve specific protocol constraints on demand. This "grounded" approach resulted in a 94.5% improvement in state transitions compared to AFLNet, demonstrating that external memory is essential for handling the complexity of network protocols.

2.6 Summary

The literature indicates a convergence toward feedback-rich, multi-agent workflows. Whether through compilation feedback (Zhang et al.), semantic validation (FalseCrashReducer), or reachability synthesis (Locus), the state-of-the-art has moved beyond simple code completion. However, gaps remain in applying these agentic principles to complex, interdependent daemons like FRRouting, where the challenge lies not just in API usage, but in managing global state and threaded contexts.

3 Methodology

This study employs an experimental research design to evaluate the efficacy of Agentic AI in generating fuzz harnesses for complex, stateful network protocols. We adapted an existing LLM-based harness generation framework, *Light-Fuzz-Gen*, to target the FRRouting (FRR) suite. This section details the target selection rationale, the limitations of standard generation approaches, and the specific architectural adaptations required to integrate agentic workflows with the FRR build system.

3.1 Target Selection: FRRouting (FRR)

We selected FRRouting (FRR) as the target system for this experiment. FRR is an open-source internet routing protocol suite used extensively in data centers, ISPs, and enterprise networks, serving as the backbone for many Software Defined Networking (SDN) environments [2].

FRR was chosen for three primary reasons:

1. **Criticality:** As a core infrastructure component implementing protocols like BGP, OSPF, and RIP, vulnerabilities in FRR have widespread impact.
2. **Complexity:** FRR utilizes a modular architecture with dedicated daemons (e.g., `bgpd`, `ospfd`, `zebra`) that rely on extensive global state and complex initialization sequences.
3. **Low Fuzzing Maturity:** Empirical analysis of the OSS-Fuzz introspector data reveals that FRR currently has limited fuzzing coverage (approximately 7.04%) [6]. Existing targets are sparse, leaving significant portions of the packet parsing logic untested.

3.2 The Challenge: Statefulness and Monolithic Daemons

Fuzzing networked applications typically involves transmitting packets over a virtual interface. However, this approach is slow and prone to packet drops by intermediate networking stacks. To bypass this, the FRR development team recommends "skipping the network" and injecting data directly into packet processing functions [3].

However, applying standard LLM harness generation to this paradigm presents two critical failures:

- **Protocol Constraints:** FRR's internal packet processing logic includes strict validation checks. To fuzz effective paths, the harness must simulate the "disarming" of checksums and bypass standard kernel interactions [3], which standard LLMs often fail to account for.
- **Architectural Isolation:** Standard tools attempt to generate standalone C files that mock external dependencies. FRR functions (e.g., `bgp_process_packet`) cannot run in isolation; they require an initialized BGP instance, peer structures, and event loops that are too complex for an LLM to hallucinate correctly.
- **Compilation Failures:** Standalone harnesses often fail to link against FRR's internal libraries (`libfrr`), resulting in undefined reference errors during compilation.

3.3 Adaptation: The "Embedded Fuzzing Block" Strategy

To overcome the limitations of standalone generation, we adapted the *Light-Fuzz-Gen* framework to support an "Embedded Fuzzing Block" strategy. Instead of generating a separate file, our agentic workflow synthesizes a code block designed to be injected directly into the daemon's `main.c` file, wrapped in `#ifdef FUZZING` directives.

This approach allows the fuzz target to reuse the daemon's actual initialization logic, memory management, and build flags, effectively bypassing the hallucination problem.

3.4 System Architecture and Pipeline

The adapted workflow proceeds in three distinct phases, combining heuristic analysis with Large Language Models to ensure structural validity.

3.4.1 Phase 1: Agentic Context Mining and Structured Output

The workflow begins with the `run_agent` process, which accepts a high-level target description. A critical challenge in LLM-driven development is ensuring the model's output is machine-readable for downstream tools. To address this, we implemented a **Strict YAML Enforcement** strategy.

The agent scans the codebase to identify relevant source files and extracts function signatures. Crucially, the system prompt explicitly constrains the output format, rejecting any response that does not conform to a pre-defined YAML schema containing fields for `target_path`, `function_signatures`, and `dependency_headers`. This converts unstructured source code into a structured intermediate representation.

3.4.2 Phase 2: Heuristic Daemon Detection

The extracted configuration is passed to the `ConfigHandler`. While the user can manually specify the target daemon, we implemented an automated fallback mechanism called the `FRRDaemonDetector`.

This module utilizes a **prefix-based heuristic analysis** to identify the correct daemon context. For example, if the mined function signatures predominantly start with `ospf_`, the detector automatically routes the configuration to the `ospfd` initialization pipeline. This ensures that the correct global state patterns (e.g., OSPF instance creation vs. BGP thread mastery) are retrieved from the `FRRPatternLibrary` without requiring manual user configuration.

3.4.3 Phase 3: Hybrid Template Generation

In the final generation phase, we reject pure "zero-shot" generation in favor of a **Hybrid Template Approach**. The system loads a static template file, `frr-embedded-fuzzing.txt`, which contains the non-negotiable C boilerplate required for the embedded block strategy.

The system dynamically injects the mined function signatures (from Phase 1) and the retrieved initialization patterns (from Phase 2) into specific placeholders within this template. The LLM is then tasked only with generating the logic to bridge these two components—calling the target function using the initialized state. This reduces the cognitive load on the model and minimizes the risk of syntax errors in the boilerplate code.

Prompt Constraints To prevent common hallucination pitfalls, we utilized a highly constrained system prompt (see Appendix A). As illustrated in our architecture, the prompt enforces strict rules:

- **Context Preservation:** It explicitly instructs the model to "reuse variable names" (e.g., `zserv`, `peer`) already declared in the manual harness, ensuring the generated code operates within the correct scope.
- **Insertion Logic:** The model is directed to place new logic *after* the core data-feeding flow to ensure that both legacy and new fuzzing behaviors execute concurrently.
- **No Standalone Files:** The prompt explicitly forbids generating full C files, requiring only the logic block wrapped in `#ifdef FUZZING` directives.

3.5 Integration and Experimental Setup

The generated harnesses were integrated into the FRR build system using the `-frr-mode` flag we added to the tool. We compiled the modified daemons using `clang` with `-fsanitize=fuzzer,address` and `-fprofile-instr-generate` to capture coverage data.

```
-g -O1 -fprofile-instr-generate -fcoverage-mapping
```

These flags were chosen to minimize optimization interference while capturing precise branch coverage.

Seed Corpus To ensure a fair comparison, both the manually written harnesses (baseline) and the LLM-generated harnesses were fuzzed for a duration of 5 hours each. We utilized a shared

seed corpus derived from the `frr-fuzz` repository [7], specifically targeting BGP message types such as `bgp-keepalive`, `bgp-update`, and `bgp-open`.The corpora were minimized every hour to maintain fuzzing efficiency.

Evaluation Metrics We quantify the performance of the generated harnesses using three standard code coverage metrics provided by LLVM:

- **Function Coverage:** The percentage of defined functions in the target daemon that were executed at least once.
- **Line Coverage:** The percentage of executable source lines hit by the fuzzer.
- **Region/Branch Coverage:** The percentage of code regions and control flow branches (e.g., `if/else` conditions) taken, which is critical for measuring the fuzzer's ability to penetrate complex protocol logic.

3.6 Hardware Environment

All experiments were conducted on Ubuntu 22.04.2 LTS with an Intel I7-1360P CPU and 16GB RAM. The LLM backend utilized for harness generation was GPT-4o via OpenAI API, chosen for its strong and proven performance in C code synthesis.

4 Evaluation and Results

4.1 Quantitative Analysis: Code Coverage

We evaluated the effectiveness of the generated harnesses by comparing them against the existing manual harnesses present in the FRR codebase. The coverage data was collected by compiling both the manual and agentic versions with `-fprofile-instr-generate -fcoverage-mapping` and running the same seed corpus for 5 hours.

4.1.1 Baseline Performance (Manual Harnesses)

First, we established a baseline by measuring the detailed coverage metrics (Function, Line, Region, and Branch) achieved by the manually written harnesses currently maintained in the FRR repository.

Daemon	Function (%)	Line (%)	Region (%)	Branch (%)
bgpd	5.77%	3.78%	3.95%	2.18%
pimd	19.42%	15.35%	18.95%	10.48%
ospfd	8.41%	6.04%	5.65%	3.09%
zebra	10.47%	6.78%	9.04%	4.03%

Table 1: Baseline Coverage Metrics: Manual Harnesses.

4.1.2 Agentic Performance (LLM-Generated Harnesses)

Next, we evaluated the harnesses generated by our adapted Light-Fuzz-Gen framework under identical experimental conditions. Table 2 presents the results.

Daemon	Function (%)	Line (%)	Region (%)	Branch (%)
bgpd	7.12%	5.32%	5.03%	3.09%
pimd	19.39%	15.52%	19.17%	10.76%
ospfd	11.38%	7.46%	6.82%	3.96%
zebra	11.46%	7.16%	9.82%	3.55%

Table 2: Experimental Coverage Metrics: LLM-Generated Harnesses.

4.1.3 Comparative Analysis

Table 3 summarizes the direct improvement in Function and Line coverage.

	Function Coverage (%)		Line Coverage (%)	
Daemon	**Manual**	**LLM-Gen**	**Manual**	**LLM-Gen**
bgpd	5.77%	**7.12%**	3.78%	**5.32%**
pimd	19.42%	19.39%	15.35%	**15.52%**
ospfd	8.41%	**11.38%**	6.04%	**7.46%**
zebra	10.47%	**11.46%**	6.78%	**7.16%**
Total	-	-	**6.7%**	**7.61%**

Table 3: Comparison of Code Coverage between Manual and LLM-Generated Harnesses.

Analysis of Improvements As illustrated in Table 3, the Agentic AI approach achieved a marginal but consistent increase in line coverage across most daemons.

- **Significant Gains:** The most significant improvement was observed in `ospfd` (+1.42%) and `bgpd` (+1.54%). This indicates that the agent successfully targeted packet parsing logic that was previously missed by manual harnesses.
- **Parity in Complexity:** For `pimd`, the results were nearly identical (15.35% vs 15.52%), suggesting that for some daemons, the harness generated by the LLM achieved only comparable performance compared to the manually written harness.
- **Overall Impact:** Crucially, the total project line coverage reached 7.61%, not only improving upon the manual baseline but also surpassing the 7.04% runtime coverage currently reported by the OSS-Fuzz introspector for the entire FRR project [6]. While numerically small, this represents hundreds of lines of critical protocol logic that are now being fuzz-tested for the first time.

4.2 Qualitative Analysis: Generated Harnesses

To understand the nature of the generated code, we examined the harnesses produced by the system (refer to Appendix B). The "Hybrid Template" approach successfully generated code that adhered to the internal API structures.

This demonstrates the model's ability to reuse global pointers (e.g., `global_peer_ptr`) extracted during the context mining phase, rather than hallucinating new, uninitialized structures.

5 Discussion

5.1 Interpretation of Coverage Limitations

While the agentic approach successfully generated compiling harnesses, the overall coverage remains low (<20% for all tested daemons). This limitation is not a failure of the LLM, but a consequence of the **Architectural Isolation** required to fuzz FRR.

To execute the harnesses safely, we had to bypass the system's network stack, socket management, and kernel interfaces. Consequently, a vast majority of the codebase—including kernel routing table logic, socket handshake orchestration, and CLI configuration subsystems—was unreachable

by design. The LLM effectively fuzzed the "data plane" (packet parsing) but could not reach the "control plane" logic that requires a full OS environment.

5.2 Limitations of the Current System

- **Dependency on Manual Patterns:** The current Phase 2 relies on a manually curated `FRRPatternLibrary`. While this prevents hallucinations, it limits the tool's scalability to new daemons that have not yet been manually analyzed, and its applicability to other networking libraries.
- **Token Cost vs. Efficiency:** The iterative "mining" and "generation" process consumes more computational resources than static template instantiation, though the cost is justified by the higher compilation success rate.

5.3 Future Work

Future research should focus on the following directions:

1. **Automated Pattern Extraction:** Replacing the manual `FRRPatternLibrary` with an agent that can dynamically infer initialization sequences from unit tests or existing binaries.
2. **Multi-YAML Integration:** Enhancing the system to consume multiple YAML configurations simultaneously, allowing the generation of harnesses that exercise multiple subsystems (e.g., parsing and route-map processing) in a single run.
3. **Automated Injection:** Developing a robust patch-application system to automatically inject the generated blocks into `main.c` without user intervention.
4. **Full Network Stack Fuzzing:** Possibly utilizing LLMs to simulate an entire network stack to emulate real-world network transmissions, allowing for fuzzing of "control plane" logic of networking libraries.
5. **Generalized Multi-Agent Orchestration:** Transitioning the current workflow into a comprehensive Multi-Agent System (MAS) that orchestrates the entire fuzzing lifecycle. Future iterations could deploy specialized agents for distinct roles—such as an *Injector Agent* to handle patch application and a *Compilation Agent* to interpret build errors—thereby automating the manual tasks of code injection and configuration management required by many libraries' build systems.

6 Conclusion

This study presented an evaluation of Agentic AI in the development of fuzz harnesses for complex network protocols. By adapting *Light-Fuzz-Gen* with a novel "Embedded Fuzzing Block" strategy, we successfully bypassed the compilation and state-management challenges inherent in the FR-Routing suite. The system achieved a total line coverage of 7.61%, outperforming the manual baseline.

These results indicate that while LLMs are not yet fully autonomous replacements for security researchers, they function as highly effective force multipliers. By shifting manual effort from writing harnesses to designing agentic workflows, we can significantly accelerate the security auditing of critical open-source infrastructure.

References

[1] Paschal C. Amusuo, Dongge Liu, Ricardo Andres Calvo Mendez, Jonathan Metzman, Oliver Chang, and James C. Davis. Falsecrashreducer: Mitigating false positive crashes in oss-fuzz-gen using agentic ai, 2025. URL https://arxiv.org/abs/2510.02185.

[2] The FRRouting Community. *FRRouting (FRR) Developer Documentation*, 2024. https://docs.frrouting.org/en/latest/.

[3] FRRouting Developers. *Fuzzing FRR: Developer's Guide*, 2024. https://docs.frrouting.org/projects/dev-guide/en/latest/fuzzing.html.

[4] Ian Hardgrove and John D. Hastings. Liblmfuzz: Llm-augmented fuzz target generation for black-box libraries, 2025. URL https://arxiv.org/abs/2507.15058.

[5] Youssef Maklad, Fares Wael, Ali Hamdi, Wael Elsersy, and Khaled Shaban. Multifuzz: A dense retrieval-based multi-agent system for network protocol fuzzing, 2025. URL https://arxiv.org/abs/2508.14300.

[6] OSS-Fuzz. Fuzz introspector report: Frrouting, 2025. Accessed Nov 2025.

[7] Quentin Young. Frr fuzzing corpus, 2019. https://github.com/qlyoung/frr-fuzz.

[8] Cen Zhang, Yaowen Zheng, Mingqiang Bai, Yeting Li, Wei Ma, Xiaofei Xie, Yuekang Li, Limin Sun, and Yang Liu. How effective are they? exploring large language model based fuzz driver generation. In *Proceedings of the 33rd ACM SIGSOFT International Symposium*

on Software Testing and Analysis, ISSTA '24, page 1223–1235. ACM, September 2024. doi: 10.1145/3650212.3680355. URL http://dx.doi.org/10.1145/3650212.3680355.

[9] Jie Zhu, Chihao Shen, Ziyang Li, Jiahao Yu, Yizheng Chen, and Kexin Pei. Locus: Agentic predicate synthesis for directed fuzzing, 2025. URL https://arxiv.org/abs/2508.21302.

A System Prompts

To ensure reproducibility, we provide the full system prompt used in the *Harness Generation* phase.

```
<system>
You are a security testing engineer generating an embedded fuzzing target for FRRouting (FRR).

The target codebase is FRRouting located at {target_repo}.

## CRITICAL: Embedded Code Format

You MUST generate code that will be embedded in {daemon}_main.c, wrapped in `#ifdef FUZZING` blocks.
This is NOT a standalone file - it will be inserted into the existing daemon main file.

The code you generate will be placed BEFORE the `#ifndef FUZZING_LIBFUZZER` block that wraps the `main()` function.

Whenever you introduce new steps inside `LLVMFuzzerTestOneInput()`, place them *after* the core data-feeding flow so both the old and new behaviors run. If additional helpers are required, implement them as `static` functions and call them from the main three functions.

Pay attention to the names of variables that are already declared in the manually defined LLVMFuzzerTestOneInput(). Reuse these variable names if you require these variables when introducing new steps.

Study the code in the existing fuzzing block to first discover how the daemon is currently being fuzzed, and the limitations of what functions can be fuzzed. Then, extend the LLVMFuzzerTestOneInput() function with the goal of exercising more functions in the fuzzing process.

## FRR-Specific Requirements

### 1. Code Structure

Generate THREE separate functions in this exact order:

1. **FuzzingInit()** - One-time daemon initialization
2. **FuzzingCreateContext()** - Context creation (daemon-specific name: {
```

```
   context_create_function})
3. **LLVMFuzzerTestOneInput()** - Main fuzzing entry point

DO NOT include `#ifdef FUZZING` wrappers - those will be added
   automatically.
DO NOT include the main() function - that already exists in the file.

### 2. Initialization Function: FuzzingInit()

Generate a `static bool FuzzingInit(void)` function that:

**Required Steps (in order):**
1. Call `frr_preinit(&{daemon}_di, 1, (char **)name)` where name is an
   array: `const char *name[] = { "{daemon}" }`
2. Initialize daemon subsystems using this sequence:
{init_sequence}

**Important Notes:**
- Use `frr_init_fast()` instead of `frr_init()` (skips signal handlers for
    fuzzing)
- For BGP: Set options to disable network operations:
  - `bgp_option_set(BGP_OPT_NO_LISTEN)` - Skip socket listening
  - `bgp_option_set(BGP_OPT_NO_FIB)` - Skip FIB updates
  - `bgp_option_set(BGP_OPT_NO_ZEBRA)` - Skip zebra communication
- For VRRP: Use `frr_init()` instead of `frr_init_fast()` (different from
   other daemons)
- Return `true` on success

### 3. Context Creation Function: {context_create_function}

Generate a `static {context_type} *{context_create_function}(void)`
   function that:

**Purpose:** Create a minimal but functional context structure for fuzzing
   .

**For BGP (FuzzingCreatePeer):**
- Create a peer structure using `peer_create()` with fake IP address
   `10.1.1.1:2001`
- Set peer state to `Established` (for packet processing)
- Set `p->bgp->rpkt_quanta = 1` (process one packet at a time)
- Set `p->status = Established`
- Set `p->as_type = AS_EXTERNAL`
```

```
- Enable all capabilities: `p->cap |= 0xFFFF`
- Activate AFI/SAFI combinations: `peer_activate(p, AFI_L2VPN, SAFI_EVPN)`
    and `peer_activate(p, AFI_IP, SAFI_MPLS_VPN)`
- Return the peer structure

**For OSPF (FuzzingCreateOspf):**
- Create fake interface: `if_get_by_name("fuzziface", 0, "default")`
- Set interface MTU: `ifp->mtu = 68`
- Create prefix: `str2prefix("11.0.2.0/24", &p)`
- Create OSPF instance: `ospf_get(0, VRF_DEFAULT_NAME, &created)`
- Set fake file descriptor: `o->fd = 69`
- Create area: `ospf_area_new(o, in)` where `in` is `0.0.0.0` (backbone
    area)
- Add interface to area: `add_ospf_interface(c, a)`
- Set interface state: `oi->state = 7` (ISM_DR - Designated Router)
- Store interface: `o->fuzzing_packet_ifp = ifp`
- Return the OSPF structure

**For VRRP (FuzzingCreateVr):**
- Create fake interface: `if_get_by_name("fuzziface", VRF_DEFAULT, "
    default")`
- Set interface MTU: `ifp->mtu = 68`
- Create prefix: `str2prefix("11.0.2.1/24", &p)`
- Create VRRP vrouter: `vrrp_vrouter_create(ifp, 10, 3)` (VRID 10,
    priority 3)
- Set FSM states: `vr->v4->fsm.state = VRRP_STATE_MASTER` and `vr->v6->fsm
    .state = VRRP_STATE_MASTER`
- Return the vrouter structure

**For Zebra (mode-dependent):**
- ZAPI mode: Create zserv client (handled in LLVMFuzzerTestOneInput)
- Netlink mode: No context needed
- Create FIFO: `fifo = stream_fifo_new()` (for ZAPI mode)

### 4. Data Feeding Mechanism: {data_feeding_mechanism}

{data_feeding_instructions}

**BGP (ringbuf mechanism):**
```c
// Reset and populate ring buffer
ringbuf_reset(p->ibuf_work);
ringbuf_put(p->ibuf_work, data, size);
```
```

```
100
101 // Validate BGP header
102 if ((size < BGP_HEADER_SIZE) || !validate_header(p)) {
103     goto done;
104 }
105
106 // Extract packet size from BGP header
107 uint16_t pktsize = 0;
108 ringbuf_peek(p->ibuf_work, BGP_MARKER_SIZE, &pktsize, sizeof(pktsize));
109 pktsize = ntohs(pktsize);
110
111 // Safety check
112 assert(pktsize <= p->max_packet_size);
113
114 // Extract complete packet if available
115 if (ringbuf_remain(p->ibuf_work) >= pktsize) {
116     struct stream *pkt = stream_new(pktsize);
117     unsigned char pktbuf[BGP_MAX_PACKET_SIZE];
118     ringbuf_get(p->ibuf_work, pktbuf, pktsize);
119     stream_put(pkt, pktbuf, pktsize);
120     p->curr = pkt;
121
122     // Process packet
123     struct event t = {};
124     t.arg = p;
125     bgp_process_packet(&t);
126 }
127 ```
128
129 **OSPF (stream mechanism):**
130 ```c
131 // Replace input buffer with fuzzed data
132 stream_free(o->ibuf);
133 o->ibuf = stream_new(MAX(1, size));
134 stream_put(o->ibuf, data, size);
135
136 // Validate IP header
137 if (size < sizeof(struct ip))
138     goto done;
139
140 struct ip *iph = (struct ip *)STREAM_DATA(o->ibuf);
141 sockopt_iphdrincl_swab_systoh(iph);  // Convert to host byte order
142 unsigned short ip_len = iph->ip_len;
```

````
143
144 // Verify packet length matches IP header
145 if (size != ip_len)
146     goto done;
147
148 // Process OSPF packet
149 ospf_read_helper(o);
150 ```
151
152 **VRRP (direct buffer mechanism):**
153 ```c
154 // Store fuzzing metadata
155 vr->v4->fuzzing_input_size = size;
156
157 // Copy data to VRRP input buffer
158 memcpy(vr->v4->ibuf, data, MIN(size, sizeof(vr->v4->ibuf)));
159
160 // Setup fake socket address
161 vr->v4->fuzzing_sa.sin_family = AF_INET;
162 inet_pton(AF_INET, "11.0.2.3", &vr->v4->fuzzing_sa.sin_addr);
163
164 // Process VRRP packet
165 struct event t;
166 t.arg = vr->v4;
167 vrrp_read(&t);
168 ```
169
170 **Zebra ZAPI mode (stream_fifo mechanism):**
171 ```c
172 // Create stream and push to FIFO
173 struct stream *s = stream_new(size + 1);
174 stream_put(s, data, size);
175 stream_fifo_push(fifo, s);
176
177 // Process ZAPI commands
178 zserv_handle_commands(zc, fifo);
179
180 // Cleanup
181 zserv_close_client(zc);
182 ```
183
184 **Zebra Netlink mode:**
185 ```c
````

```
// Direct netlink message processing
netlink_fuzz(data, size);
```

5. Entry Point: LLVMFuzzerTestOneInput()

Generate `int LLVMFuzzerTestOneInput({function_signature_params})` that:

Required Structure:
1. Check `FuzzingInitialized` static flag (one-time initialization)
2. If not initialized:
   - Call `FuzzingInit()`
   - Set `FuzzingInitialized = true`
   - Call `{context_create_function}()` and store result in global variable
3. Get context (persistent for libFuzzer):
   ```c
   #ifdef FUZZING_LIBFUZZER
       ctx = FuzzingContext;  // Reuse existing context
   #else
       ctx = FuzzingContext;  // AFL mode (same for now)
   #endif
   ```
4. Feed fuzzed data using {data_feeding_mechanism} (see examples above)
5. Validate headers: {validation_requirements}
6. Call processing function: {processing_function}
7. After the primary processing call, inject additional logic that explicitly exercises **each** function from the `{functions}` list (create helper invocations or call chains as needed). These calls should build on the state produced by the existing harness rather than resetting everything.
7. Return 0

Important Notes:
- Use `const uint8_t *data` for most daemons
- Use `uint8_t *data` (non-const) for VRRP
- Always check minimum size before processing
- Use `goto done;` for early exits
- Label `done:` before return statement

6. Static Variables

Declare these static variables at the top (before functions):

````
```c
static bool FuzzingInitialized;
static {context_type} *FuzzingContext;
```

**Note:** For VRRP, use:
```c
bool FuzzingInitialized;
struct vrrp_vrouter *FuzzingVr;
```

## Functions to Fuzz

{functions}

## Function Signatures

{function_signatures}

## Additional Information

{additional}

## Validation Requirements

{validation_requirements}

## Processing Function

{processing_function}

## Code Quality Requirements

1. **Compilation**: Code MUST compile against FRR codebase without errors
2. **No NULL pointers**: All pointers must be initialized before use
3. **Variable declarations**: If using `goto`, declare all variables
   before the `goto` statement
4. **Type matching**: Ensure all types match function signatures exactly
5. **Memory safety**: Use proper bounds checking (e.g., `MIN(size, sizeof(
   buffer))`)
6. **Error handling**: Check return values and minimum sizes before
   processing
7. **Indentation**: Use tabs (not spaces) for indentation (FRR standard)
````

````
## Example Structure

Here's the expected structure (without #ifdef wrappers):

```c
// Static variables
static bool FuzzingInitialized;
static {context_type} *FuzzingContext;

// Initialization function
static bool FuzzingInit(void) {
    // ... initialization code ...
    return true;
}

// Context creation function
static {context_type} *{context_create_function}(void) {
    // ... context creation code ...
    return context;
}

// Main fuzzing entry point
int LLVMFuzzerTestOneInput({function_signature_params}) {
    // One-time initialization check
    if (!FuzzingInitialized) {
        FuzzingInit();
        FuzzingInitialized = true;
        FuzzingContext = {context_create_function}();
    }

    // Get context
    #ifdef FUZZING_LIBFUZZER
        {context_type} *ctx = FuzzingContext;
    #else
        {context_type} *ctx = FuzzingContext;
    #endif

    // Data feeding and processing
    // ... (use appropriate mechanism from examples above) ...

done:
    return 0;
````
````

```
}
```

```

## Critical Reminders

1. DO NOT include '#ifdef FUZZING' - it will be added automatically
2. DO NOT include 'main()' function - it already exists
3. DO use the exact function names: 'FuzzingInit', '{context_create_function}', 'LLVMFuzzerTestOneInput'
4. DO preserve the existing harness semanticsadd new logic without removing the canonical BGP/OSPF/PIM/VRRP/Zebra steps that are already present in FRR.
5. DO follow the data feeding mechanism examples exactly
6. DO include proper validation before processing
7. DO use 'goto done;' for early exits with a 'done:' label before return
8. DO ensure all variables are declared before any 'goto' statements
9. DO use tabs for indentation (FRR coding standard)

Generate the three functions now, following the structure and examples above.

<solution>

...
```

Listing 1: Full System Prompt for FRR Harness Generation

B Generated Harness Example

Below is an example of a fully generated, compilable embedded block for the bgpd daemon.

```
#ifdef FUZZING
#include "lib/ringbuf.h"

int LLVMFuzzerTestOneInput(const uint8_t *data, size_t size);

static bool FuzzingInit(void) {
        int bgp_port = BGP_PORT_DEFAULT;
        struct list *addresses = list_new();
        int no_fib_flag = 0;
        int no_zebra_flag = 0;
        int skip_runas = 0;
```

```
        int instance = 0;
        int buffer_size = BGP_SOCKET_SNDBUF_SIZE;
        struct listnode *node;

        addresses->cmp = (int (*)(void *, void *))strcmp;

        const char *name[] = { "bgpd" };

        frr_preinit(&bgpd_di, 1, (char **) name);

        /* Initialize basic BGP datastructures */
        bgp_master_init(frr_init_fast(), buffer_size, addresses);
        bm->port = bgp_port;
        bgp_option_set(BGP_OPT_NO_LISTEN);

        bgp_option_set(BGP_OPT_NO_FIB);
        bgp_option_set(BGP_OPT_NO_ZEBRA);
        bgp_error_init();
        // bgp_vrf_init();
        bgp_init((unsigned short)instance);


        /* Create a default instance and peer */
        struct bgp *b;
        as_t as = 65001;
        bgp_get(&b, &as, "default", BGP_INSTANCE_TYPE_DEFAULT, NULL,
                ASNOTATION_UNDEFINED);

        return true;
}

/*
 * Create peer structure that we'll use as the receiver for fuzzed packets
    .
 *
 * state
 *    BGP FSM state to create the peer in
 */
static struct peer *FuzzingCreatePeer(int state)
{
        union sockunion su;
        sockunion_init(&su);
        inet_pton(AF_INET, "10.1.1.1", &su.sin.sin_addr);
```

```
        su.sin.sin_family = AF_INET;
        su.sin.sin_port = 2001;
        struct peer *p = peer_create(&su, NULL, bgp_get_default(), 65000,
           65001,
                                     AS_UNSPECIFIED, NULL, 1, NULL);
        p->bgp->rpkt_quanta = 1;
        p->status = state;
        p->as_type = AS_EXTERNAL;

        /* set all flags */
        afi_t afi;
        safi_t safi;
        p->cap |= 0xFFFF;
        FOREACH_AFI_SAFI(afi, safi) {
                SET_FLAG(p->af_cap[afi][safi], 0x3FFF);
        }

        peer_activate(p, AFI_L2VPN, SAFI_EVPN);
        peer_activate(p, AFI_IP, SAFI_MPLS_VPN);

        return p;
}

static struct peer *FuzzingPeer;

static bool FuzzingInitialized;

static void FuzzingExerciseConfig(struct peer *peer, const uint8_t *data,
                                  size_t size)
{
        struct bgp *bgp;
        uint32_t seed = 0;
        struct in_addr cluster_id;
        as_t confed_as;
        char as_buf[32];
        bool no_listen_was_set;

        if (!peer)
                return;

        bgp = peer->bgp;
        if (!bgp)
                return;
```

```
96
97          if (size >= sizeof(seed))
98                  memcpy(&seed, data + (size - sizeof(seed)), sizeof(seed));
99          else {
100                 for (size_t i = 0; i < size; i++)
101                         seed = (seed << 8) | data[i];
102         }
103
104         if (!seed)
105                 seed = 0x01020304;
106
107         cluster_id.s_addr = htonl(seed);
108         bgp_cluster_id_set(bgp, &cluster_id);
109         bgp_cluster_id_unset(bgp);
110
111         confed_as = (seed % 64512U) + 1;
112         snprintf(as_buf, sizeof(as_buf), "%u", (unsigned int)confed_as);
113         bgp_confederation_id_set(bgp, confed_as, as_buf);
114         bgp_confederation_id_unset(bgp);
115
116         bgp_default_local_preference_unset(bgp);
117         bgp_listen_limit_unset(bgp);
118
119         no_listen_was_set = bgp_option_check(BGP_OPT_NO_LISTEN);
120         bgp_option_unset(BGP_OPT_NO_LISTEN);
121         if (no_listen_was_set)
122                 bgp_option_set(BGP_OPT_NO_LISTEN);
123 }
124
125 int LLVMFuzzerTestOneInput(const uint8_t *data, size_t size)
126 {
127         if (!FuzzingInitialized) {
128                 FuzzingInit();
129                 FuzzingInitialized = true;
130                 /* See comment below */
131                 FuzzingPeer = FuzzingCreatePeer(Established);
132         }
133         //make_route_maps();
134         /*
135          * In the AFL standalone case, the peer will already be created
              for us
136          * before __AFL_INIT() is called to speed things up. We can't pass
              it
```

```
137           * as an argument because the function signature must match
                libFuzzer's
138           * expectations, so it's saved in a global variable. Even though
                we'll
139           * be exiting the program after each run, we still destroy the
                peer
140           * because that increases our coverage and is likely to find
                memory
141           * leaks when pointers are nulled that would otherwise be "in-use
                at
142           * exit".
143           *
144           * In the libFuzzer case, we can either create and destroy it each
                 time
145           * to fuzz single packet rx, or we can keep the peer around, which
                 will
146           * fuzz a long lived session. Of course, as state accumulates over
147           * time, memory usage will grow, which imposes some resource
148           * constraints on the fuzzing host. In practice a reasonable
                server
149           * machine with 64gb of memory or so should be able to fuzz
150           * indefinitely; if bgpd consumes this much memory over time, that
151           * behavior should probably be considered a bug.
152           */
153          struct peer *p;
154  #ifdef FUZZING_LIBFUZZER
155          /* For non-persistent mode */
156          // p = FuzzingCreatePeer();
157          /* For persistent mode */
158          p = FuzzingPeer;
159  #else
160          p = FuzzingPeer;
161  #endif /* FUZZING_LIBFUZZER */
162
163          ringbuf_reset(p->ibuf_work);
164          ringbuf_put(p->ibuf_work, data, size);
165
166          int result = 0;
167          unsigned char pktbuf[BGP_MAX_PACKET_SIZE];
168          uint16_t pktsize = 0;
169
170          /*
171           * Simulate the read process done by bgp_process_reads().
```

```
172             *
173             * The actual packet processing code assumes that the size field
                    in the
174             * BGP message is correct, and this check is performed by the i/o
                    code,
175             * so we need to make sure that remains true for fuzzed input.
176             *
177             * Also, validate_header() assumes that there is at least
178             * BGP_HEADER_SIZE bytes in ibuf_work.
179             */
180            if ((size < BGP_HEADER_SIZE) || !validate_header(p)) {
181                    goto done;
182            }
183
184            ringbuf_peek(p->ibuf_work, BGP_MARKER_SIZE, &pktsize, sizeof(
                   pktsize));
185            pktsize = ntohs(pktsize);
186
187            assert(pktsize <= p->max_packet_size);
188
189            if (ringbuf_remain(p->ibuf_work) >= pktsize) {
190                    struct stream *pkt = stream_new(pktsize);
191                    ringbuf_get(p->ibuf_work, pktbuf, pktsize);
192                    stream_put(pkt, pktbuf, pktsize);
193                    p->curr = pkt;
194
195                    struct event t = {};
196                    t.arg = p;
197                    bgp_process_packet(&t);
198            }
199
200            FuzzingExerciseConfig(p, data, size);
201
202    done:
203            //peer_delete(p);
204
205            return 0;
206    };
207    #endif /* FUZZING */
```

Listing 2: Full Generated Embedded Block